Specificația Cerințelor Software
Echipa 14

1.1 Tematica Aleasă

Proiectul implementează un server concurent care oferă servicii de procesare a datelor prin trei mecanisme de transport diferite. Funcționalitățile principale includ: alocarea de identificatori unici pentru clienți, operații de echo și concatenare a șirurilor de caractere, operații aritmetice (negare, adunare) și, ca element central avansat, rezolvarea de sisteme liniare de ecuații Ax = b folosind biblioteca LAPACKE și eliminarea Gaussiană paralelă cu procese fork(). Clientul permite formularea de ecuații de ordin 1, 2 sau 3 în format algebric, parsarea lor în matrice și transmiterea către server pentru rezolvare.

1.2 Referințe

IEEE 830-1998 - Standard pentru specificații de cerințe software

OpenAPI 3.0.3 — Specificația utilizată pentru descrierea formală a API-ului

POSIX.1-2008 — Standard pentru interfețele de sistem utilizate (pthreads, sockets, fork)

gSOAP 2.8 — Toolkit pentru generarea automată a codului SOAP din fișiere header C

LAPACKE — Interfața C pentru LAPACK, utilizată pentru descompunerea LU cu pivotare parțială

2.1 Perspectiva Produsului

Serverul PCD este o aplicație standalone care rulează ca proces daemon pe o mașină Linux. La pornire, procesul principal (threeds.c) citește configurația din fișierul server.cfg folosind biblioteca libconfig, aplică eventualele suprascriiri din variabilele de mediu și argumentele din linia de comandă, apoi lansează trei thread-uri POSIX, fiecare responsabil de un canal de comunicație diferit.

2.2 Componentele Principale

Componentă	Fișier Sursă	Descriere
Punct de intrare	threeds.c	Parsare CLI (getopt_long), încărcare configurație, lansare thread-uri
Modul configurație	config.c / config.h	Parsare server.cfg cu libconfig, valori implicite, suprascriere CLI
Server INET TCP	inetds2.c	Protocol binar propriu, multiplexare cu select()
Server SOAP	soapds.c	Serviciu SOAP document/literal generat cu soapcpp2 din sclient.h
Server UNIX Domain	unixds.c	Socket UNIX SOCK_STREAM cu funcționalitate echo
Protocol binar	proto.c / proto.h	Serializare/deserializare mesaje, antet 12 octeți, byte order
Rezolvator sisteme liniare	solver.c / solver.h	LAPACKE_dgesv si eliminare Gaussiană paralelă
Client ecuații	equation_type1_client.c	Parsare ecuații algebrice, construire matrice A/b, comunicare cu serverul
Definiții SOAP	sclient.h	Header gSOAP cu structurile și semnăturile operațiilor WS

Componentă	Fișier Sursă	Descriere
Fișier configurare	server.cfg	Configurare porturi, socket, logging, solver
Specificație API	openapi_pcd.yaml	Documentație OpenAPI 3.0.3 pentru protocolul binar și SOAP

2.3 Funcționalități ale Sistemului

F1 — Alocare ID client (OPR_CONNECT): Serverul generează un identificator unic combinând timestamp-ul Unix (16 biți inferiori) cu un contor atomic (12 biți)

F2 — Echo (OPR_ECHO): Returnează clientului exact șirul de caractere primit, util pentru verificarea conectivității și integrității protocolului.

F3 — Concatenare (OPR_CONC): Primește două șiruri de caractere separate și returnează concatenarea lor cu spațiu separator.

F4 — Negare (OPR_NEG): Primește un număr întreg și returnează opusul său

F5 — Adunare (OPR_ADD): Primește doi operanzi întregi într-un singur mesaj MultiInt și returnează suma lor.

F6 — Rezolvare sistem liniar (OPR_SOLVE): Primește matricea A și vectorul b. Rezolvă sistemul Ax = b prin LAPACKE sau eliminare Gaussiană paralelă cu procese fork(). Returnează vectorul soluție x sau un cod de eroare.

F7 — Deconectare (OPR_BYE): Semnalează încheierea sesiunii; serverul închide file descriptor-ul asociat fără a trimite răspuns.

F8 — Serviciu SOAP echivalent: Operațiile ns__connect, ns__echo, ns__concat, ns__adder și ns__bye oferă funcționalitate similară prin protocolul SOAP.

F9 — Canal UNIX Domain Socket: Oferă un serviciu simplu de echo prin socket UNIX SOCK_STREAM, accept() și select().3. Arhitectura Sistemului

3.1 Diagrama Arhitectural

Proiectul este organizat pe un model multi-threaded cu un punct central de intrare care lansează trei thread-uri dedicate. Fiecare thread gestionează un canal de comunicație independent:

Thread 1 — UNIX Domain Socket (unix_main): Ascultă pe calea configurată, acceptă conexiuni SOCK_STREAM și oferă funcționalitate echo folosind select() pentru multiplexare.

Thread 2 — INET TCP (inet_main): Ascultă pe portul TCP configurat, multiplexează conexiunile cu select() și procesează mesajele binare conform protocolului definit în proto.h.

Thread 3 — SOAP (soap_main): Ascultă pe portul SOAP configurat folosește API-ul gSOAP (soap_bind, soap_accept, soap_serve) și procesează cererile SOAP

3.2 Fluxul de Pornire

Etapa 1 — Citire variabile de mediu: PCD_CONFIG și PCD_LOG_LEVEL

Etapa 2 — Parsare argumente CLI cu getopt_long(): --config, --port, --soap-port, --unix-socket, --log-level, --help.

Etapa 3 — Încărcare fișier de configurare via config_load() cu biblioteca libconfig. Cheile lipsă primesc valori implicite.

Etapa 4 — Suprascriere cu argumente CLI prin config_apply_args(). Prioritatea este: CLI > fișier .cfg > variabile.

Etapa 5 — Afișare configurație finală cu config_print() și lansare thread-uri cu pthread_create().

3.3 Configurația Sistemului

Parametru	Secțiune .cfg	Tip	Implicit	Descriere
inet_port	server.inet_port	uint16_t	18081	Port TCP pentru protocolul binar
soap_port	server.soap_port	uint16_t	18082	Port pentru serviciul SOAP
unix_socket	server.unix_socket	char[256]	/tmp/unixds	Cale socket UNIX domain
backlog	server.backlog	int	10	Dimensiune coadă listen()
reuse_addr	server.reuse_addr	int	1	Flag SO_REUSEADDR
log_level	logging.level	int	1	0=erori, 1=info, 2=debug
log_dest	logging.destination	char[256]	stderr	Destinație log
demo_string	demo.input_string	char[256]	Hello..	Șir pentru demo libconfig
demo_int1	demo.input_int1	int	42	Primul întreg demo
demo_int2	demo.input_int2	int	58	Al doilea întreg demo
solver_workers	solver.workers	int	4	Număr procese fork() pentru solver
solver_threshold	solver.serial_threshold	int	32	Pragul n sub care se folosește calea serială

4. Structura Mesajelor și a Protocolului (FCE)

4.1 Formatul Cadrului de Comunicație (FCE) — Protocol Binar TCP

Toate mesajele transmise pe canalul INET TCP încep cu un antet fix de 12 octeți.

4.1.1 Antetul Mesajului (msgHeaderType)

Câmp	Tip	Offset (octeți)	Dimensiune	Descriere
msgSize	int32_t	0	4 octeți	Dimensiunea totală a mesajului current.
clientID	int32_t	4	4 octeți	ID-ul clientului
opID	int32_t	8	4 octeți	Codul operației cerute

4.1.2 Codurile Operațiilor (OPR_*)

Cod	Constantă	Descriere	Payload Cerere	Payload Răspuns
0	OPR_CONNECT	Alocare ID client	SingleInt	SingleInt
1	OPR_ECHO	Echo șir de caractere	SingleInt + octeti	SingleInt+ octeti
2	OPR_CONC	Concatenare două șiruri	2× [SingleInt+octeti]	SingleInt + octeti
3	OPR_NEG	Negare întreg	SingleInt	SingleInt
4	OPR_ADD	Adunare doi întregi	MultiInt	SingleInt(a+b)
5	OPR_BYE	Deconectare	Doar antet	Fără răspuns
6	OPR_SOLVE	Rezolvare Ax=b	SolveReqHdr + A + b	SolveRspHdr [+ x]

4.1.3 Tipuri de Mesaje

singleIntMsgType — 16 octeți: antet (12) + msgIntType (4). Folosit pentru OPR_CONNECT, OPR_ECHO (lungime șir), OPR_NEG și răspunsuri întregi.

multiIntMsgType — 20 octeți: antet (12) + msg2IntType (8). Folosit pentru OPR_ADD (doi operanzi într-un singur mesaj).

Mesaj șir de caractere — Transmis în două etape: mai întâi un SingleInt cu lungimea șirului (fără terminator nul), apoi octeții șirului. Funcțiile readSingleString() și writeSingleString() gestionează acest format compus.

4.1.4 Structura Mesajelor OPR_SOLVE

solveRequestHeaderType — 16 octeți: antet msgHeader + n . Urmat de n×n valori double pentru matricea A și n valori double pentru vectorul b.

Segment	Dimensiune	Format	Descriere
solveRequestHeader	16 octeți	msgHeader + int32 n	Antet + ordinul sistemului
Matricea A	n×n × 8 octeți	double[], row-major, nativ	Coeficienții sistemului
Vectorul b	n × 8 octeți	double[], nativ	Termenii liberi

solveResponseHeaderType — 20 octeți: antet msgHeader + status + n. Dacă status = 0, urmează n valori double reprezentând vectorul soluție x.

Segment	Dimensiune	Format	Descriere
solveResponseHeader	20 octeți	msgHeader + status + n	Antet + cod stare + ordin
Vectorul x	n × 8 octeți	double[], nativ	Soluția (doar dacă status=0)

Cod Status	Semnificație
0	Succes — vectorul soluție x este prezent în răspuns
-1	Matrice singulară — sistemul nu are soluție unică
-2	Eroare de alocare memorie pe server
-3	Eroare internă IPC (comunicare între procese eșuată)

4.2 Protocolul SOAP (gSOAP)

Serviciul SOAP este generat automat de utilitarul soapcpp2 din fișierul header sclient.h. Stilul utilizat este document/literal cu namespace-ul http://tempuri.org/ns.xsd. Serviciul rulează pe portul 18082 (implicit) și este accesibil la adresa 127.0.0.1.

4.2.1 Operații SOAP

Operație SOAP	Semnătură	Cerere	Răspuns	Descriere
ns__connect	int ns__connect	Fără parametri	long clientID	Alocă un ID unic (
ns__echo	int ns__echo	string echoRequest	string echo	Returnează 'echo ' + șirul primit
ns__concat	int ns__concat	concatStruct	string concat	Returnează op2 + ' ' + op1
ns__adder	int ns__adder	addStruct	long adder	Returnează op1 + op2
ns__bye	int ns__bye	byeStruct	Răspuns gol	Încheie sesiunea

4.2.2 Structuri SOAP

ns__concatType Conține câmpurile: long id char *op1, char *op2

ns__addType (addStruct): Conține câmpurile: long id , long op1, long op2

ns__byeType (byeStruct): Conține un singur câmp: long id

4.3 Canalul UNIX Domain Socket

Canalul UNIX Domain Socket oferă un serviciu simplu de echo. Socket-ul de tip SOCK_STREAM este creat la calea configurată (implicit /tmp/unixds). Thread-ul unix_main() folosește select() pentru multiplexarea conexiunilor. Datele primite de la client sunt trimise înapoi identic. Dimensiunea maximă a buffer-ului de recepție este de 512 octeți.

Acest canal nu implementează protocolul binar msgHeader — este destinat în principal pentru testarea mecanismului de socket-uri UNIX și nu expune operațiile OPR_*.

5.1 Calea Serială

Pentru sisteme de ordin n<SOLVER_SERIAL_THRESHOLD sau când solver_workers = 1, rezolvatorul apelează direct funcția LAPACKE_dgesv() din biblioteca LAPACKE. Aceasta efectuează o descompunere LU cu pivotare parțială

5.2 Calea Paralelă

Pentru sisteme de ordin n ≥ serial_threshold cu mai mulți workeri, rezolvatorul folosește eliminare Gaussiană paralelă bazată pe procese copil (fork()). Cele n rânduri ale matricei augmentate [A|b] sunt împărțite în num_workers benzi contigue. Fiecare proces copil deține o bandă pe toată durata calculului.

Comunicare prin pipe-uri: Pentru fiecare worker se creează două pipe-uri (down[] și up[]). Pipe-ul down transmite date de la procesul părinte către copil, iar pipe-ul up în sens invers. La fiecare pas k de eliminare, procesul care deține rândul pivot îl trimite prin up[], părintele îl redistribuie tuturor copiilor prin down[], iar fiecare copil efectuează eliminarea pe rândurile proprii.

Substituție inversă: După n pași de eliminare, copiii trimit rândurile finale înapoi la părinte, care colectează matricea superior triunghiulară și efectuează substituția inversă pentru a obține vectorul soluție x.

Limite: Numărul maxim de workeri este SOLVER_MAX_WORKERS = 16. Dacă num_workers > n, se reduce automat la n.

6. Clientul de Ecuații (equation_type1_client.c)

6.1 Descriere

Clientul permite formularea de sisteme de ecuații liniare de ordin 1, 2 sau 3 în format algebric natural, le parsează în matricea coeficienților A și vectorul termenilor liberi b, se conectează la server prin protocolul binar TCP, execută handshake-ul OPR_CONNECT, apoi trimite cererea OPR_SOLVE și afișează soluția primită.

6.2 Utilizare

```
./equation_type1_client -n 1 "2x+4=6"
./equation_type1_client -n 2 "2x+3y=7" "x+2y=4"
./equation_type1_client -n 3 "x+y+z=6" "2x-y+z=3" "x+2y-z=2"
```

6.3 Formate de Ecuații Acceptate

Ordin	Format	Exemple
1	ax+b=c | ax-b=c | ax=c | x+b=c | x-b=c | x=c	2x+4=6, x=7
2	ax+by=c | ax-by=c	2x+3y=7, -x+2y=4
3	ax+by+cz=d	x+y+z=6, 2x-y+z=5

6.4 Fluxul de Comunicare Client-Server

1. Clientul se conectează via TCP la host:port (implicit 127.0.0.1:18081).
2. Trimite cerere OPR_CONNECT cu clientID=0 , deci primește ID alocat.
3. Trimite cerere OPR_SOLVE cu matricea A și vectorul b, deci primește status și vectorul soluție x.
4. Afișează rezultatul, trimite OPR_BYE și închide conexiunea.

7.1 Informații Generale

Specificația OpenAPI documentează formal cele două canale principale de comunicație ale serverului: protocolul binar TCP/INET și serviciul SOAP.

Proprietate	Valoare
Versiune OpenAPI	3.0.3
Titlu API	PCD Server – Protocol API
Versiune API	1.0.0
Server TCP	http://localhost:18081
Server SOAP	http://localhost:18082 (SOAP document)

7.2 Endpoint-uri INET TCP (Pseudo-REST)

Endpoint	Metodă	Operație	Descriere	Schema
/inet/connect	POST	OPR_CONNECT (0)	Alocare clientID	ConnectRequest → ConnectResponse
/inet/echo	POST	OPR_ECHO (1)	Echo șir	EchoRequest → EchoResponse
/inet/concat	POST	OPR_CONC (2)	Concatenare	ConcatRequest → ConcatResponse
/inet/negate	POST	OPR_NEG (3)	Negare întreg	NegRequest → NegResponse
/inet/add	POST	OPR_ADD (4)	Adunare	AddRequest → AddResponse
/inet/solve	POST	OPR_SOLVE (6)	Rezolvare Ax=b	SolveRequest → SolveResponse
/inet/bye	POST	OPR_BYE (5)	Deconectare	ByeRequest → fără răspuns

7.3 Endpoint-uri SOAP

Endpoint	Metodă	Operație	Descriere	Schema
/soap/connect	POST	ns__connect	Alocare clientID SOAP	→ SoapConnectResponse
/soap/echo	POST	ns__echo	Echo cu prefix	SoapEchoRequest → SoapEchoResponse
/soap/concat	POST	ns__concat	Concatenare (ordine inversă)	SoapConcatRequest → SoapConcatResponse
/soap/adder	POST	ns__adder	Adunare doi întregi	SoapAdderRequest → SoapAdderResponse
/soap/bye	POST	ns__bye	Deconectare SOAP	SoapByeRequest → 204

7.4 Scheme de Date OpenAPI

Scheme Protocol Binar

MsgHeader: Obiect cu câmpurile msgSize , clientID și opID. Reprezintă antetul fix de 12 octeți transmis în big-endian.

ConnectRequest/Response: Cererea conține header + value=0. Răspunsul conține header + clientID alocat

EchoRequest/Response: Header + length + message. Răspunsul are structură identică.

ConcatRequest/Response: Cererea conține header + două șiruri consecutive. Răspunsul returnează un singur șir concatenat.

NegRequest/Response: Header + value. Răspunsul conține result = -value.

AddRequest/Response: Cererea conține header + operand1 + operand2. Răspunsul conține result = operand1 + operand2.

SolveRequest: Header + n + matrixA + vectorB. Valorile double sunt în byte order nativ.

SolveResponse: Header + status + n + solutionX .

ByeRequest: Doar header-ul. Serverul închide socket-ul fără răspuns.

Scheme SOAP

SoapConnectResponse: Obiect cu câmpul clientID .

SoapEchoRequest/Response: Cerere cu message. Răspuns cu result = ' echo ' + message.

SoapConcatRequest/Response: Cerere cu id, op1, op2. Răspuns cu result = op2 + ' ' + op1.

SoapAdderRequest/Response: Cerere cu id, op1, op2. Răspuns cu result = op1 + op2 .

SoapByeRequest: Obiect cu id. Sesiune încheiată, răspuns 204.

8.1 Performanță

Serverul trebuie să suporte conexiuni simultane pe toate cele trei canale fără degradare perceptibilă. Rezolvatorul paralel trebuie să demonstreze speedup pentru sisteme de ordin n ≥ 32 cu cel puțin 2 workeri. Calea serială LAPACKE_dgesv este preferată pentru sisteme mici datorită overhead-ului de fork()/pipe().

8.2 Fiabilitate și Robustețe

Toate apelurile de sistem (socket, bind, listen, accept, send, recv, fork, pipe, read, write) sunt verificate pentru erori. Eroarea de protocol (clientID negativ, opID invalid) conduce la închiderea conexiunii. Funcțiile WRITE_ALL și READ_ALL din solver.c asigură transfer complet al datelor pe pipe-uri

8.3 Securitate

Serviciul SOAP ascultă doar pe 127.0.0.1 (loopback). Nu există mecanism de autentificare, aplicația este destinată mediilor de test și dezvoltare. Dimensiunea maximă a ordinului n pentru OPR_SOLVE este limitată la 4096 pentru a preveni alocări excesive de memorie.

8.4 Portabilitate

Aplicația este compilată cu -std=c11 și -D_POSIX_C_SOURCE=200809L, asigurând conformitatea cu standardul C11 și interfețele POSIX. Dependențele externe sunt: libpthread, libgsoap, libconfig, liblapacke, libopenblas.

8.5 Mentenabilitate

Codul este compilat cu flag-uri stricte de avertizare (-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wformat=2 -Wnull-dereference) și este supus analizei statice cu clang-tidy (ținta make tidy). Configurația este centralizată într-o singură structură globală (g_cfg) accesibilă din toate modulele.

9.1 Dependințe de Sistem

```
sudo apt-get install -y gsoap libgsoap-dev libconfig-dev liblapacke-dev libopenblas-dev
clang-tidy
```

9.2 Comenzi de Build

Comandă	Descriere
make deps	Instalează toate dependințele necesare
make all	Compilează serverul și clientul de ecuații
make serverds	Compilează doar serverul multi-threaded
make equation_type1_client	Compilează doar clientul de ecuații
make tidy	Rulează clang-tidy pe toate fișierele sursă
make clean	Șterge artefactele de build

9.3 Rulare

```
./serverds                                          - cu valori implicite
./serverds -c /path/to/server.cfg -p 9000 -l 2      -cu opțiuni CLI
PCD_CONFIG=alt.cfg PCD_LOG_LEVEL=2 ./serverds       - cu variabile de mediu
./equation_type1_client -n 2 "2x+3y=7" "x+2y=4"    - client ecuații
```